

Assignment 3

This assignment focuses on implementing a Recurrent Neural ODE model for a regression task on time series data. You will work with a synthetic dataset of spirals.

Dataset

The dataset consists of four variations, each representing different levels of observation completeness, with missing points randomly selected for each time series:

- **spirals_100.npz**: The full dataset, containing spiral time series with all 100 data points regularly sampled at equal intervals.
- **spirals_75.npz**: A reduced version where each spiral retains only 75 randomly selected data points.
- **spirals_50.npz**: A further reduced dataset with spirals containing 50 data points.
- **spirals_25.npz**: The most sparse dataset, with only 25 data points per spiral.

While the time series in **spirals_100.npz** are regularly sampled, the other datasets feature irregularly sampled data points. This irregularity is naturally handled by neural ODEs, whereas traditional RNN models often require additional preprocessing or workarounds to manage such data.

The dataset **spirals_n.npz**, **n** is the number of data points (100, 75, 50, or 25), contains the following arrays:

- **data_train**: Training data (shape **(1000, n, 3)**).
- **data_validation**: Validation data (shape **(1000, n, 3)**).
- **data_test**: Test data (shape **(1000, n, 3)**).
- The three channels of these data correspond to the time values **t**, and **x** and **y** components of the spirals.
- **alpha_train** and **alpha_validation** (shape **(1000, 1)**): Corresponding **α** values for each split except for the test split, which is to be predicted as part of this assignment.

Tasks

Your goal is to build a latent variable model that can predict the parameter α given a spiral trajectory. Choose and implement one of GRU-ODE, ODE-RNN, ODE-LSTM (see Table 1 of <https://arxiv.org/abs/2502.09885> for explicit equations.). The model should use its recurrent nature to process the input sequence `data` and update its hidden state. Either the entire hidden trajectory or the final hidden state can then be used to predict `alpha`.

Train the model to predict `alpha_train` using the `data_train` array. Then use your trained model and `data_test` to predict `alpha_test`.

Each dataset is independent and should be used exclusively for its corresponding predictions. For example, you must use `spirals_100.npz` to predict `alpha` values for `data_test` in `spirals_100.npz`. Similarly, `spirals_25.npz` should only be used to predict `alpha` values for `data_test` in `spirals_25.npz`.

Submission

- Documented code for your model, training loop, and evaluation. Include a plot of training and validation loss trajectories.
- To complete the task, predict `alpha_test` values using `txy_test` for each dataset and save the predictions as a NumPy `.npz` file named `{your_name}.npz`. The file should contain four keys: `'100'`, `'75'`, `'50'`, and `'25'`, each corresponding to the predictions for the respective datasets, stored as arrays of shape `(10000, 1)`. After creating the file, validate your submission by editing `submission_validation.py` to set `fname = '{your_name}.npz'`, ensuring both the script and the `.npz` file are in the same directory. Run the validation script to confirm correctness, and once validated, submit the `{your_name}.npz` file.

Example code that generates the prediction archive:

```
import numpy as np

model = {'100': model_100
        , '75': model_75
        , '50': model_50
        , '25': model_25}

alpha_pred = {}
for perc in [100, 75, 50, 25]:
    archive = np.load(f'spirals_{perc}.npz')
```

```
data_test = archive['data_test']
alpha_pred[str(perc)] = model[str(perc)](data_test)

np.savez('umur.npz', **alpha_pred)
```

- Present your code and findings in class on 25.11.2025